compromise a user, so the safest approach is to listen only for HTTPS requests, not HTTP requests. This is the best option for API apps.

NOTE It would be safest to take this same approach for your browser apps, but unfortunately, browsers currently default to the HTTP versions of apps by default.

You can disable HTTP requests for your application by setting the URLs for your app to include only https:// requests, using `ASPNETCORE_URLS` or another approach, as described in chapter 27. Setting

```
ASPNETCORE_URLS=https://*:5001
```

would ensure that your app serves only HTTPS requests on port 5001 and won't handle HTTP connections at all. This protects your clients, as they can't incorrectly make HTTP requests, and it may even make things simpler on your side, as you don't need to add the HTTP redirection middleware.

HTTPS is one of the most basic requirements for adding security to your application these days. It can be tricky to set up initially, but once you're up and running, you can largely forget about it, especially if you're using SSL/TLS termination at a reverse proxy.

Unfortunately, most other security practices require rather more vigilance to ensure that you don't accidentally introduce vulnerabilities into your app as it grows and develops. In the next chapter we'll look at several common attacks, learn how ASP.NET Core protects you, and see a few things you need to watch out for.

Summary